HW7

Zhen Wu

Prob. 1

```r
# Define the objective function
f <- function(x) {
  return((x^3 / 3) - (x^2 / 2) - x - 1)
}

# Implementation of the Golden Section Search
golden_section_search <- function(func, a, b, tol = 1e-6) {
  phi <- (sqrt(5) - 1) / 2

  # Calculate initial internal points
  x1 <- b - phi * (b - a)
  x2 <- a + phi * (b - a)

  # Initial evaluations
  f1 <- func(x1)
  f2 <- func(x2)

  iteration <- 0
  while ((b - a) > tol) {
    iteration <- iteration + 1
    if (f1 < f2) {
      b <- x2
      x2 <- x1
      f2 <- f1
      x1 <- b - phi * (b - a)
      f1 <- func(x1)
    } else {
      a <- x1
      x1 <- x2
      f1 <- f2
      x2 <- a + phi * (b - a)
      f2 <- func(x2)
    }
  }

  x_opt <- (a + b) / 2
  return(list(x_min = x_opt, f_min = func(x_opt), iterations = iteration))
}

# Execute the search
result <- golden_section_search(f, 1, 2, 1e-6)
```

```r
# Output results
cat("Optimal x:", format(result$x_min, nsmall = 8), "\n")
```

```
## Optimal x: 1.61803409
```

```r
cat("Minimum value f(x):", format(result$f_min, nsmall = 8), "\n")
```

```
## Minimum value f(x): -2.51502832
```

```r
cat("Total iterations:", result$iterations, "\n")
```

```
## Total iterations: 29
```

Prob. 2

(a)

To demonstrate that the function $f(x) = e^{-x} - \cos(x)$ is unimodal and possesses a unique global minimizer on the interval $[0, 1]$, we examine its second-order properties within the framework of mathematical analysis.We begin by computing the first and second derivatives of the objective function:

$$f'(x) = -e^{-x} + \sin(x)$$

$$f''(x) = e^{-x} + \cos(x)$$

On the closed interval $x \in [0, 1]$, the exponential term e^{-x} is strictly positive, satisfying $e^{-1} \leq e^{-x} \leq 1$.Simultaneously, since the interval $[0, 1]$ is a subset of $[0, \pi/2]$, the trigonometric term $\cos(x)$ remains strictly positive, with $\cos(1) \leq \cos(x) \leq 1$.Consequently, the second derivative is strictly positive for all x in the domain:

$$f''(x) = e^{-x} + \cos(x) > 0, \quad \forall x \in [0, 1]$$

This strict positivity of the second derivative implies that $f(x)$ is strictly convex on $[0, 1]$.In optimization theory, a strictly convex function defined on a convex set is inherently unimodal.To ensure the minimizer lies within the interior of the interval, we evaluate the first derivative at the boundaries:

$$f'(0) = -e^0 + \sin(0) = -1 < 0$$

$$f'(1) = -e^{-1} + \sin(1) \approx -0.3678 + 0.8414 = 0.4736 > 0$$

Since $f'(x)$ is continuous and changes sign from negative to positive over $[0, 1]$, the Intermediate Value Theorem guarantees the existence of a point $x^* \in (0, 1)$ such that $f'(x^*) = 0$.Given the strict convexity, this critical point is the unique global minimizer of the function.

(b)

```r
# Define the objective function f(x)
f <- function(x) {
  return(exp(-x) - cos(x))
}

# Implementation of the Golden Section Search
# We use a standard tolerance epsilon = 1e-6
golden_section <- function(a, b, epsilon = 1e-6) {
  phi <- (sqrt(5) - 1) / 2

  # Determine initial interior points
  x1 <- b - phi * (b - a)
  x2 <- a + phi * (b - a)
```

```r
  f1 <- f(x1)
  f2 <- f(x2)

  # Count evaluations: 2 initial + 1 per iteration
  eval_count <- 2

  while ((b - a) > epsilon) {
    if (f1 < f2) {
      b <- x2
      x2 <- x1
      f2 <- f1
      x1 <- b - phi * (b - a)
      f1 <- f(x1)
    } else {
      a <- x1
      x1 <- x2
      f1 <- f2
      x2 <- a + phi * (b - a)
      f2 <- f(x2)
    }
    eval_count <- eval_count + 1
  }

  x_star <- (a + b) / 2
  return(list(minimizer = x_star, value = f(x_star), evaluations = eval_count))
}

# Execute search on [0, 1]
result <- golden_section(0, 1)

cat("The unique global minimizer x* is:", result$minimizer, "\n")
```

```
## The unique global minimizer x* is: 0.5885327
```

```r
cat("The minimum value f(x*) is:", result$value, "\n")
```

```
## The minimum value f(x*) is: -0.2766149
```

```r
cat("Number of function evaluations used:", result$evaluations, "\n")
```

```
## Number of function evaluations used: 31
```

(c)

To analyze the relationship between the interval size and the computational effort, we consider the interval reduction property of the algorithm.The length of the interval after n iterations, denoted by L_n, follows a geometric progression based on the golden ratio $\phi \approx 0.618$:

$$L_n = L_0 \cdot \phi^n$$

To achieve a target precision ϵ, the number of iterations n must satisfy the following inequality:

$$L_0 \cdot \phi^n < \epsilon \implies n > \frac{\ln(\epsilon/L_0)}{\ln(\phi)}$$

Since each iteration after the initialization requires exactly one new function evaluation, the total number of evaluations is $N = n + 1$.If we reduce the size of the initial interval L_0 while keeping the tolerance ϵ constant,

the ratio ϵ/L_0 increases.Because $\ln(\phi)$ is a negative constant, an increase in ϵ/L_0 results in a smaller required value for n.Therefore, the number of function evaluations strictly decreases as the initial search interval shrinks.This confirms that narrowing the initial search space based on prior knowledge effectively reduces the computational cost of the Golden Section Method.

Prob. 3

(a)

To determine the order of convergence α and the rate c, we analyze the limit of the ratio of successive absolute errors as $t \to \infty$.Algorithm I: The absolute error is given by $\epsilon^{(t)} = (0.3)^t$.We evaluate the limit for $\alpha = 1$:

$$\lim_{t\to\infty} \frac{|\epsilon^{(t+1)}|}{|\epsilon^{(t)}|^1} = \lim_{t\to\infty} \frac{(0.3)^{t+1}}{(0.3)^t} = \lim_{t\to\infty} 0.3 = 0.3$$

Since the limit is a constant $c \in (0, 1)$ when $\alpha = 1$, Algorithm I exhibits linear convergence with $\alpha = 1$ and $c = 0.3$.Algorithm II: The absolute error is given by $\epsilon^{(t)} = (0.5)^{3^t}$.We seek an α such that the ratio approaches a constant:

$$\frac{|\epsilon^{(t+1)}|}{|\epsilon^{(t)}|^\alpha} = \frac{(0.5)^{3^{t+1}}}{((0.5)^{3^t})^\alpha} = \frac{(0.5)^{3\cdot 3^t}}{(0.5)^{\alpha\cdot 3^t}} = (0.5)^{(3-\alpha)3^t}$$

To ensure the expression does not depend on t as $t \to \infty$, we set the exponent to zero by choosing $\alpha = 3$:

$$\lim_{t\to\infty} (0.5)^{(3-3)3^t} = \lim_{t\to\infty} (0.5)^0 = 1$$

Thus, Algorithm II exhibits cubic convergence with $\alpha = 3$ and $c = 1$.

(b)

For Algorithm III, the absolute error is defined as $\epsilon^{(t)} = \frac{1}{t}$.We compute the limit assuming a linear order of convergence ($\alpha = 1$):

$$\lim_{t\to\infty} \frac{|\epsilon^{(t+1)}|}{|\epsilon^{(t)}|^1} = \lim_{t\to\infty} \frac{\frac{1}{t+1}}{\frac{1}{t}} = \lim_{t\to\infty} \frac{t}{t+1} = 1$$

Strictly based on the standard constraints for linear convergence, Algorithm III does not satisfy the definition.While the order $\alpha = 1$, the convergence rate c must satisfy $0 < c < 1$ for a sequence to be considered linearly convergent.When $c = 1$, the convergence is classified as sublinear, meaning the error decreases more slowly than any geometric progression.

(c)

The relative convergence criterion $\frac{|x^{(t+1)} - x^{(t)}|}{|x^{(t)}|} < tol$ is dangerous for Algorithm III because the step size vanishes significantly faster than the actual error.Assuming x^* is small or normalized such that we can focus on the absolute difference in the numerator, the step size for Algorithm III is:

$$|x^{(t+1)} - x^{(t)}| = |\epsilon^{(t+1)} - \epsilon^{(t)}| = \left|\frac{1}{t+1} - \frac{1}{t}\right| = \frac{1}{t(t+1)} \approx \frac{1}{t^2}$$

In this case, the step size decreases at an order of $O(t^{-2})$, while the actual error $\epsilon^{(t)}$ only decreases at an order of $O(t^{-1})$.As t increases, the difference between successive iterations becomes extremely small, potentially triggering the stopping criterion *tol* long before the actual error $\epsilon^{(t)}$ has reached an acceptable level.This creates a "false convergence" scenario where the software terminates because the progress is slow, not because the solution is accurate.The following R code demonstrates this disparity by comparing the error and the step size over iterations.

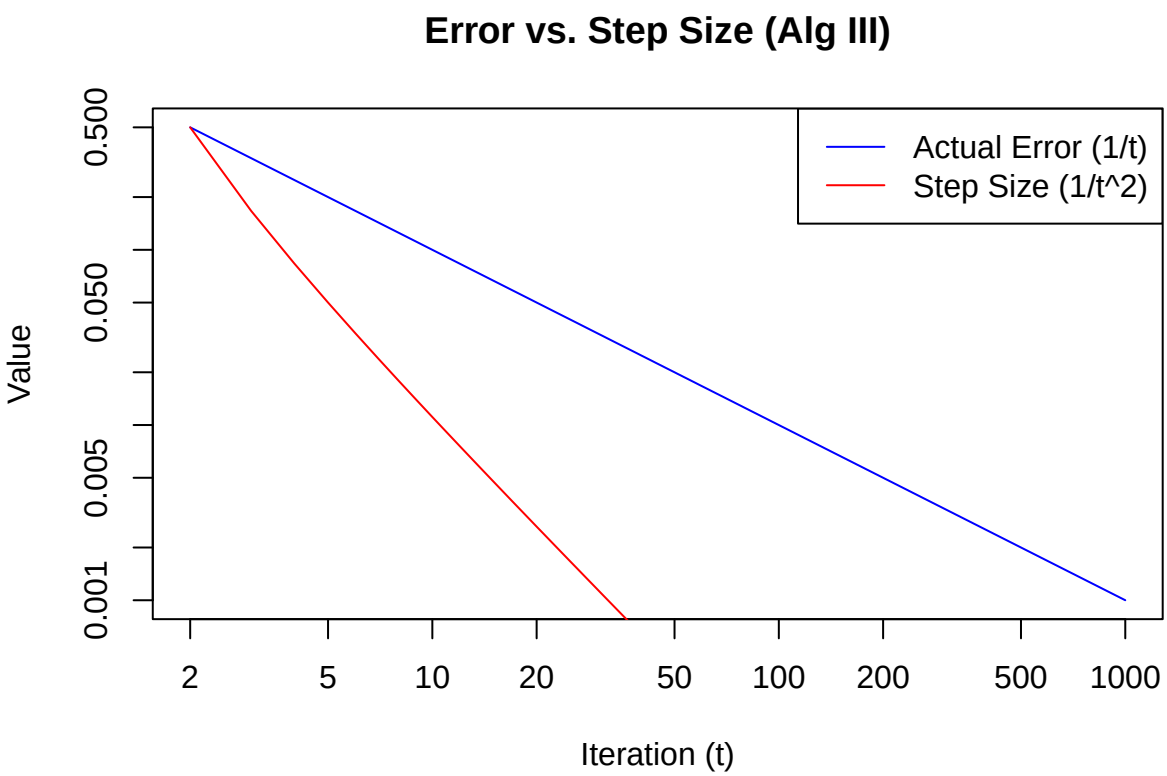
```r
# Parameters
iterations <- 1:1000
epsilon_t <- 1 / iterations
step_size <- abs(diff(epsilon_t))

# Create a comparison table
comparison <- data.frame(
  t = iterations[-1],
  Error = epsilon_t[-1],
  Step_Size = step_size
)

# Display first few rows to show the gap
print(head(comparison, 10))
```

```
##     t      Error   Step_Size
## 1   2 0.50000000 0.500000000
## 2   3 0.33333333 0.166666667
## 3   4 0.25000000 0.083333333
## 4   5 0.20000000 0.050000000
## 5   6 0.16666667 0.033333333
## 6   7 0.14285714 0.023809524
## 7   8 0.12500000 0.017857143
## 8   9 0.11111111 0.013888889
## 9  10 0.10000000 0.011111111
## 10 11 0.09090909 0.009090909
```

```r
# Visualization
plot(comparison$t, comparison$Error, type = "l", col = "blue", log = "xy",
     xlab = "Iteration (t)", ylab = "Value", main = "Error vs. Step Size (Alg III)")
lines(comparison$t, comparison$Step_Size, col = "red")
legend("topright", legend = c("Actual Error (1/t)", "Step Size (1/t^2)"),
       col = c("blue", "red"), lty = 1)
```



Prob. 4

(a)

To rewrite the function $f(x) = 2x_1^2 + x_2^2 - 2x_1x_2 - 4x_1 + 6x_2 + 5$ into the standard matrix format $f(x) = \frac{1}{2}x^T Ax - b^T x + c$, we must identify the coefficients corresponding to the quadratic, linear, and constant terms.For a symmetric matrix A, the quadratic component is given by the expansion:

$$\frac{1}{2}x^T Ax = \frac{1}{2}\begin{pmatrix} x_1 & x_2 \end{pmatrix}\begin{pmatrix} a_{11} & a_{12} \\ a_{12} & a_{22} \end{pmatrix}\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \frac{1}{2}a_{11}x_1^2 + a_{12}x_1x_2 + \frac{1}{2}a_{22}x_2^2$$

By comparing this to the quadratic terms $2x_1^2 + x_2^2 - 2x_1x_2$, we determine the elements of A:

$$\frac{1}{2}a_{11} = 2 \implies a_{11} = 4$$

$$\frac{1}{2}a_{22} = 1 \implies a_{22} = 2$$

$$a_{12} = -2$$

Thus, the symmetric matrix A is:

$$A = \begin{pmatrix} 4 & -2 \\ -2 & 2 \end{pmatrix}$$

Next, we align the linear terms $-b^T x = -b_1x_1 - b_2x_2$ with $-4x_1 + 6x_2$, yielding:

$$-b_1 = -4 \implies b_1 = 4$$

$$-b_2 = 6 \implies b_2 = -6$$

Therefore, the vector b and the scalar c are:

$$b = \begin{pmatrix} 4 \\ -6 \end{pmatrix}, \quad c = 5$$

(b)

The gradient of a quadratic function in the form $f(x) = \frac{1}{2}x^T Ax - b^T x + c$ is derived as follows:

$$\nabla f(x) = \nabla \left(\frac{1}{2}x^T Ax - b^T x + c \right) = Ax - b$$

A stationary point (and potential minimizer) occurs where the gradient vanishes, leading to the linear system:

$$Ax = b \implies \begin{pmatrix} 4 & -2 \\ -2 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 4 \\ -6 \end{pmatrix}$$

Solving this system yields:

$$4x_1 - 2x_2 = 4$$

$$-2x_1 + 2x_2 = -6$$

Summing the two equations gives $2x_1 = -2$, which implies $x_1 = -1$.Substituting x_1 back into the second equation results in $2 + 2x_2 = -6$, so $x_2 = -4$.The minimizer is $x^* = (-1, -4)^T$.In practice, for solving such systems, the Cholesky decomposition is often preferred over direct inversion or simple Gaussian elimination when A is symmetric and positive-definite.This is because Cholesky decomposition is computationally more efficient (requiring approximately half the operations of LU decomposition) and numerically more stable for these specific matrix structures.The following R code solves the system and verifies the minimizer:

```r
# Define matrix A and vector b
A <- matrix(c(4, -2, -2, 2), nrow = 2, byrow = TRUE)
b <- c(4, -6)

# Solve for x using the solve() function (uses LU/QR internally)
x_star <- solve(A, b)

# Verify using Cholesky decomposition
L <- chol(A) # Note: R returns the upper triangular factor
y <- solve(t(L), b)
x_chol <- solve(L, y)

cat("The minimizer x* is:\n")
```

```
## The minimizer x* is:
```

```r
print(x_star)
```

```
## [1] -1 -4
```

(c)

To determine if the matrix A is positive-definite, we evaluate its leading principal minors:

$$\Delta_1 = |4| = 4 > 0$$

$$\Delta_2 = \det(A) = (4)(2) - (-2)(-2) = 8 - 4 = 4 > 0$$

Since all leading principal minors are strictly positive, A is positive-definite (denoted $A \succ 0$).This property is critical because it implies that the Hessian of the function, $H = \nabla^2 f(x) = A$, is positive-definite everywhere in $\mathbb{R}^2$.A function with a constant positive-definite Hessian is strictly convex.For strictly convex functions, any critical point where $\nabla f(x) = 0$ is guaranteed to be the unique global minimum.Without this property (e.g., if A were indefinite), the solution to $Ax = b$ could represent a saddle point rather than a minimum, which would be insufficient for optimization tasks like least squares or maximum likelihood estimation.